

Testes (Contrato e Integração)

Justificativa: A robustez da migração depende de testes abrangentes. Testes de contrato evitam regressões inesperadas entre o consumidor (monólito) e o provedor (novo microsserviço), enquanto testes de integração garantem que o provedor responda exatamente conforme o contrato especificado.

Resultado esperado: Conjunto de testes de contrato (validação da OpenAPI) e testes de integração (via ferramentas como Supertest) "verdes", assegurando a conformidade e a estabilidade da API.

Testes de Contrato (Onde e Como)

A garantia da integridade e consistência de uma API é fundamental em arquiteturas de microsserviços. Os testes de contrato desempenham um papel crucial ao verificar se a implementação do serviço adere estritamente à sua especificação OpenAPI. No nosso repositório, disponibilizamos exemplos práticos desses testes, especificamente em `packages/auth-service/test/contract/openapi.spec.ts`, que servem como modelo para outros serviços.

O princípio fundamental desses testes é validar a presença e a estrutura de elementos esperados no arquivo `openapi.yaml` de cada pacote. Isso inclui verificar se os `paths` (endpoints) e `schemas` (modelos de dados) definidos estão presentes e acessíveis, evitando que alterações acidentais ou intencionais na especificação passem despercebidas, o que poderia levar a quebras na comunicação entre os serviços.

Título sugerido (issue/PR): Implementar testes de contrato para [users-service](#) (`openapi.yaml`)

Descrição / Prompt:

Por favor, implemente testes de contrato para o pacote [users-service](#) que verifiquem a presença e estrutura mínima do `openapi.yaml` do serviço de usuários.

Requisitos:

- Local do teste: criar o arquivo
 - `packages/users-service/test/contract/openapi.spec.ts`
- Ferramenta: Jest (usar o setup de testes já presente no repositório; os outros pacotes usam TypeScript nos testes).
- Dependência para ler/parsear YAML: `yaml` (adicionar como devDependency em [users-service](#) se ainda não existir).
- O teste deve:
 - Ler [openapi.yaml](#).
 - Fazer o parse com `yaml` e transformar em objeto JS.
 - Asserções obrigatórias:
 - `spec.paths['/users']` está definido.
 - `spec.paths['/users'].post` está definido.
 - `spec.components.schemas.CreateUser` está definido.
 - `spec.components.schemas.User` está definido.

- Opcional (fortemente recomendado): verificar que a referência do `requestBody` do `POST /users` aponta para `#/components/schemas/CreateUser` (ou que o schema de request contenha os campos esperados).
- a. Retornar mensagens de erro claras para cada asserção (ex.: "CreateUser schema not found").

Critérios de aceite:

- O teste passa localmente (rodando o runner de testes do repo) e falha explicitamente se qualquer checagem acima não for satisfeita.
- Submissão via PR contendo:
 - O novo arquivo de teste `packages/users-service/test/contract/openapi.spec.ts`.
 - Se necessário, atualização de `package.json` para incluir `yaml` em `devDependencies` (ou instrução no README do pacote sobre instalação).
 - Breve nota no PR descrevendo como rodar apenas os testes de contrato (ex.: comando ou filtro do jest).

Validação de Contrato OpenAPI na Prática

Continuando nossa discussão sobre testes de contrato, esta seção detalha a implementação prática para validar a especificação OpenAPI de um microsserviço. O objetivo é assegurar que o arquivo `openapi.yaml` – que serve como a fonte da verdade para o contrato da API – contenha todos os elementos cruciais conforme esperado. Utilizaremos o exemplo fornecido para ilustrar a estrutura básica de um teste de contrato, focando na leitura e validação da presença de `paths` e `schemas`.

O padrão para esses testes envolve duas etapas principais:

- **Leitura do `openapi.yaml`:** O arquivo de especificação é lido e parseado para um objeto JavaScript/TypeScript utilizando uma biblioteca como `yaml`.
- **Validação da presença de elementos:** Asserções são feitas para garantir que endpoints (`paths`) e modelos de dados (`schemas`) essenciais estejam definidos na especificação.

Este é um exemplo direto que demonstra como verificar a existência de um endpoint específico (`/users`) e de schemas fundamentais (`CreateUser`) no arquivo OpenAPI, prevenindo que alterações não intencionais ou falhas na documentação do contrato passem despercebidas.

```
import fs from 'fs';
import YAML from 'yaml';

// Lê e parseia o arquivo openapi.yaml do serviço
const raw = fs.readFileSync('packages/users-service/openapi.yaml', 'utf8');
const spec = YAML.parse(raw);
```

```
// Valida a presença de paths e schemas específicos
expect(spec.paths['/users']).toBeDefined();
expect(spec.paths['/users'].post).toBeDefined(); // Adicionado para ser
// mais específico
expect(spec.components.schemas.CreateUser).toBeDefined();
expect(spec.components.schemas.User).toBeDefined(); // Adicionado para ser
// mais abrangente
```

Este trecho de código exemplifica a base para um teste de contrato robusto, garantindo que o serviço de usuários mantenha sua integridade e que seu contrato seja sempre válido e completo, essencial para a comunicação entre microsserviços.

Como Executar os Testes de Contrato

Com a estrutura dos testes de contrato estabelecida, o próximo passo crucial é saber como executá-los para validar a conformidade da sua especificação OpenAPI. Existem duas abordagens principais para rodar esses testes, ambas utilizando o Jest, que é a ferramenta de teste configurada no repositório. O uso do comando `--runInBand` é recomendado para garantir que os testes sejam executados sequencialmente, evitando possíveis conflitos de recurso ou estado, especialmente em um ambiente de monorepo.

```
cd packages/users-service
npm install --save-dev jest ts-jest @types/jest yml
npx ts-jest config:init
```

1 A partir da Raiz do Monorepo (Recomendado)

Esta é a forma mais comum e recomendada de executar os testes, pois permite gerenciar e rodar testes de múltiplos pacotes a partir de um único ponto. Utilize o `npx jest` e especifique o caminho completo para o arquivo de teste desejado.

```
# a partir do root do repositório
npx jest packages/users-service/test/contract/openapi.spec.ts --runInBand
```

2 A partir do Diretório do Pacote

Alternativamente, você pode navegar diretamente para o diretório do pacote do serviço de usuários e executar o Jest a partir de lá. Esta abordagem é útil se você estiver focando exclusivamente no desenvolvimento e teste de um único serviço.

```
cd packages/users-service
npx jest test/contract/openapi.spec.ts --runInBand
```

Ao seguir estas instruções, você garantirá que os testes de contrato sejam executados de maneira eficiente, fornecendo feedback imediato sobre qualquer inconsistência entre a implementação do serviço e sua especificação OpenAPI. Manter uma rotina de execução desses testes é fundamental para a saúde e a estabilidade de uma arquitetura de microsserviços.

Gerenciando Dependências de Testes e Alternativas para Validação OpenAPI

A execução bem-sucedida dos testes de contrato é intrinsecamente ligada a um ambiente de desenvolvimento robusto e configurado adequadamente. Esta seção detalha como solucionar obstáculos comuns relacionados às dependências de testes e oferece uma alternativa expedita para a validação manual de sua especificação OpenAPI, essencial para manter a saúde do seu contrato de API.

Dependências de Testes (Se ocorrerem Erros)

Caso se depare com mensagens como "No tests found" ou falhas na transformação de arquivos TypeScript durante a execução dos testes de contrato, isso frequentemente aponta para dependências ausentes ou configurações incompletas. Mesmo que o Jest e o ts-jest já estejam configurados na raiz do monorepo, pode ser necessário instalá-los diretamente no diretório do seu serviço específico (e.g., `packages/users-service`) para garantir que o ambiente de teste seja autossuficiente.

```
# dentro do pacote, se preferir instalar localmente
cd packages/users-service
npm install --save-dev jest ts-jest @types/jest yaml

# opcional: inicializar configuração ts-jest (gera jest config mínima)
npx ts-jest config:init
```

A instalação local destas dependências assegura um isolamento adequado, prevenindo conflitos com outras configurações do monorepo. O comando `npx ts-jest config:init` é particularmente útil para iniciar uma configuração mínima do Jest, permitindo personalizações futuras conforme as necessidades do projeto.

Alternativa Simples (Validação sem TypeScript)

Para cenários de depuração ágil ou quando a validação da presença de chaves específicas em seu arquivo YAML é urgente e dispensa um ambiente de teste completo (ou a compilação TypeScript), uma solução simplificada está disponível via script Node.js direto. Esta abordagem oferece um feedback instantâneo sobre a estrutura básica do seu `openapi.yaml`.

Interpretação e Otimização dos Testes de Contrato

Depois de executar seus testes de contrato, é crucial entender como interpretar os resultados e quais passos seguir para otimizar seu fluxo de trabalho. Esta seção detalha as saídas esperadas do Jest e oferece sugestões para aprimorar sua estratégia de testes, garantindo a robustez da sua API.

Interpretando os Resultados do Jest

- **Sucesso:** Quando todos os testes são aprovados, o Jest exibirá a mensagem `PASS` para o arquivo de teste executado, acompanhada de um resumo conciso indicando que todos os testes foram concluídos com êxito. Isso valida que a especificação OpenAPI está em conformidade com o contrato esperado.
- **Falha:** Em caso de falha, a saída do Jest será detalhada, apontando exatamente qual asserção não foi atendida. Por exemplo, você pode ver erros indicando a ausência de um `path` específico como `/users`, ou a falta de definição de um `schema` crucial. Se a mensagem `No tests found` aparecer, isso geralmente indica um problema na configuração do ambiente de teste ou na instalação das dependências, conforme abordado na seção anterior.

Sugestões para Otimização e Fluxo de Trabalho

Para maximizar a eficácia dos seus testes de contrato e integrar-os de forma mais fluida ao seu processo de desenvolvimento, considere as seguintes sugestões:

Alternativa sem Dependência de TypeScript

Se sua preferência é por um ambiente de execução de testes que não dependa do TypeScript, é possível criar uma versão em JavaScript puro do seu teste de contrato (`openapi.spec.js`). Esta abordagem utilizaria `require('yaml')` e `fs` diretamente, oferecendo uma camada adicional de flexibilidade e simplificando o processo para equipes que operam predominantemente em JavaScript. Caso haja interesse, podemos detalhar a implementação dessa alternativa.

Integração Contínua (CI)

Adicione um `job` simples à sua Pipeline de Integração Contínua (CI) que execute os testes de contrato. Este `job` deve ser configurado para rodar o comando de teste apropriado (por exemplo, `npx jest packages/users-service/test/contract/openapi.spec.ts --runInBand`) e falhar o pipeline caso os testes de contrato não sejam aprovados. Essa prática é fundamental para garantir que qualquer regressão ou alteração que viole o contrato da API seja detectada precocemente, antes que o código chegue ao ambiente de produção.

```
node -e "const fs=require('fs'); const YAML=require('yaml'); const spec=YAML.parse(fs.readFileSync('packages/users-service/openapi.yaml','utf8')); console.log(Object.keys(spec.paths | | {}));"
```

Este comando executa um script Node.js que lê, parseia e exibe as chaves de `paths` do seu arquivo OpenAPI, servindo como uma ferramenta de diagnóstico rápido para verificar a integridade da sua especificação sem a sobrecarga de um ciclo de teste completo.

Gerar Client e Server Stubs com OpenAPI Generator

O OpenAPI Generator é uma ferramenta poderosa que automatiza a criação de artefatos de código a partir de uma especificação OpenAPI (anteriormente conhecida como Swagger). Ele desempenha um papel crucial na manutenção da consistência entre a documentação da API e sua implementação, além de acelerar significativamente o ciclo de desenvolvimento.



O que é e para que serve

O OpenAPI Generator é uma ferramenta que lê uma especificação OpenAPI e gera automaticamente clientes de API, servidores stubs, SDKs e documentação, garantindo a consistência entre a especificação e o código.



Geração de Cliente

Permite criar uma biblioteca completa para o consumidor da API. Isso inclui métodos tipados, modelos de dados e lógica de serialização/desserialização, facilitando a integração de novas aplicações.



Geração de Server Stub

Cria um esqueleto de projeto de servidor, com rotas, modelos e controladores vazios, permitindo que os desenvolvedores foquem na implementação da lógica de negócio sem se preocupar com a estrutura inicial da API.

A utilização dessa ferramenta não só economiza tempo, mas também minimiza erros manuais, assegurando que todas as partes envolvidas no desenvolvimento de uma API trabalhem com uma base comum e atualizada. Veja abaixo exemplos de comandos para gerar código:

```
# gerar um client TypeScript com axios
npx @openapitools/openapi-generator-cli generate \
  -i packages/users-service/openapi.yaml \
  -g typescript-axios \
  -o generated/users-client

# gerar um server stub Node.js/Express
npx @openapitools/openapi-generator-cli generate \
  -i packages/users-service/openapi.yaml \
  -g nodejs-express-server \
  -o generated/users-server
```

Estes comandos são apenas um ponto de partida. O OpenAPI Generator suporta uma vasta gama de linguagens e frameworks, oferecendo flexibilidade para se adaptar a praticamente qualquer ecossistema de desenvolvimento, desde front-ends a back-ends e microsserviços.

Configuração e Utilização Avançada do OpenAPI Generator

Além dos métodos diretos de execução, o OpenAPI Generator oferece flexibilidade através de alternativas que se adaptam a diferentes ambientes de desenvolvimento, como o uso via Docker, evitando a necessidade de instalação global. Compreender a estrutura dos artefatos gerados e como integrá-los ao seu fluxo de trabalho é crucial para otimizar o desenvolvimento e manter a consistência da sua API.

Alternativas de Execução para o Gerador

Para desenvolvedores que preferem não instalar a CLI do OpenAPI Generator globalmente ou que trabalham em ambientes containerizados, o Docker oferece uma maneira elegante de executar a ferramenta. Esta abordagem garante um ambiente consistente e isolado para a geração de código.

```
docker run --rm -v ${PWD}:/local openapitools/openapi-generator-cli generate \
  -i /local/packages/users-service/openapi.yaml -g typescript-axios -o /local/generated/users-client
```

Este comando executa o gerador dentro de um contêiner Docker, mapeando o diretório de trabalho atual (`${PWD}`) para `/local` dentro do contêiner. Isso permite que o gerador acesse o arquivo `openapi.yaml` e grave os artefatos gerados diretamente no seu sistema de arquivos local, sem a necessidade de pré-instalação.

O Conteúdo dos Artefatos Gerados

Os artefatos produzidos pelo OpenAPI Generator são projetados para acelerar o desenvolvimento, fornecendo uma base sólida para a interação com sua API, seja como cliente ou como implementação de servidor.

Client (typescript-axios)

- Modelos de dados (interfaces/classes) tipados para as estruturas de requisição e resposta.
- Serviços/APIs com funções para cada endpoint da sua especificação.
- Um arquivo `README` detalhado com instruções de uso e lista de dependências (como o Axios).
- Lógica de serialização e desserialização já implementada.

Server Stub (nodejs-express-server)

- Uma estrutura de rotas baseada na sua especificação OpenAPI.
- Esqueletos de controladores (métodos vazios) prontos para receber a lógica de negócio.
- Modelos de dados que representam os schemas da API.
- Instruções básicas para iniciar o servidor stub.

Guia de Uso Rápido

A integração dos artefatos gerados em seu projeto segue um processo simples, projetado para ser intuitivo e rápido.

1. Gere o artefato utilizando um dos comandos mencionados acima, direcionando a saída para o diretório `generated/` em seu projeto.
2. Acesse o diretório do cliente ou servidor stub gerado e instale suas dependências:

```
cd generated/users-client  
npm install
```

Após a instalação das dependências, você estará pronto para importar e utilizar o cliente em seu projeto front-end ou começar a implementar a lógica do seu servidor, aproveitando a tipagem e a estrutura que o OpenAPI Generator proporciona. Este processo economiza tempo, minimiza erros e garante que seu código esteja sempre em sincronia com a sua especificação OpenAPI.

Utilizando os Artefatos Gerados e Melhores Práticas

Com os clientes e stubs de servidor devidamente gerados pelo OpenAPI Generator, o próximo passo é integrá-los e utilizá-los em seus projetos. Esta seção detalha como consumir a API usando o client gerado, como iniciar a implementação no server stub e as melhores práticas para garantir a reprodutibilidade e a qualidade do código.

Consumindo a API com o Client Gerado (TypeScript)

O client gerado para TypeScript, utilizando Axios, oferece uma interface tipada e intuitiva para interagir com a API. Isso significa que você pode chamar os endpoints da sua API como métodos de uma classe, com autocompletar e validação de tipos, o que é um grande benefício para a produtividade e a robustez do código.

```
import { UsersApi } from './generated/users-client';

const api = new UsersApi({ basePath: 'http://localhost:3010' });
await api.createUser({ email: 'a@b.com', name: 'Aluno' });
```

Este exemplo demonstra a facilidade de uso: basta instanciar a API e chamar o método correspondente ao endpoint desejado, passando os dados necessários. O client cuida de todos os detalhes de requisição HTTP, serialização e desserialização, permitindo que você se concentre na lógica de negócio.

Implementando a Lógica no Server Stub

Para o server stub, a ideia é fornecer uma base sobre a qual sua lógica de negócio será construída. Após gerar o server stub (por exemplo, `nodejs-express-server`), você encontrará um esqueleto de projeto com rotas e controladores vazios. O processo geralmente envolve:

1. Abrir o diretório `generated/users-server`.
2. Instalar as dependências do projeto (`npm install`).
3. Implementar os controladores, preenchendo os métodos vazios com a lógica que responderá às requisições dos endpoints.
4. Rodar o servidor conforme as instruções no arquivo `README.md` gerado, que geralmente inclui um comando como `npm start`.

Esta abordagem permite que você desenvolva a implementação do servidor em paralelo com o desenvolvimento do client, garantindo que ambos estejam sempre alinhados com a especificação OpenAPI.

Boas Práticas e Cuidados Didáticos no Uso do OpenAPI Generator

Versionamento da Especificação e do Generator

Para garantir a reprodutibilidade dos artefatos gerados, é crucial versionar tanto a especificação OpenAPI (`openapi.yaml`) quanto a versão exata do OpenAPI Generator utilizada. Isso evita surpresas com mudanças de comportamento em versões futuras da ferramenta.

Gerenciamento do Código Gerado

Decida se o código gerado será commitado para o controle de versão. Em ambientes de aula, pode ser útil commitar um exemplo fixo em um diretório como `examples/generated/`. Em projetos reais, é comum regenerar o código em um pipeline de CI/CD, evitando que o código gerado polua o histórico do repositório.

Revisão e Adaptação do Código

Sempre revise o código gerado antes de usá-lo em produção. Embora a ferramenta seja excelente, ela gera código genérico que pode precisar de adaptações para se alinhar aos padrões de estilo, convenções de código ou necessidades específicas do seu projeto. Ajustes são normalmente necessários.

Compatibilidade OpenAPI 3.0 vs. 3.1

Esteja ciente de que alguns geradores podem ter melhor suporte para OpenAPI 3.0 do que para 3.1. Se encontrar problemas na geração, uma estratégia temporária pode ser converter sua especificação para 3.0 ou verificar se há uma versão mais recente do generator que ofereça suporte a 3.1.

Seguir essas diretrizes ajudará a maximizar os benefícios do OpenAPI Generator, mantendo seus projetos consistentes e eficientes.